SolvencyInsight

AI-Powered Financial Distress Prediction System

Technical Documentation & Team Contributions

Team Member	Primary Responsibility
Rohan	Machine Learning Module & Backend Architecture
Jonathan	Machine Learning Module & Backend Development
Neha	Data Preparation & Frontend Development
Mariya	Frontend Development & UI/UX
All Members	System Integration & Testing

1. Project Overview

SolvencyInsight is an enterprise-grade financial distress prediction platform that combines traditional financial analysis (Altman Z-Score) with modern machine learning (XGBoost with SHAP explainability) to predict corporate bankruptcy risk. The system provides real-time risk assessment with interpretable AI explanations.

Core Capabilities:

* **Bankruptcy Prediction:** XGBoost classifier trained on 12 financial ratios with probability smoothing for realistic predictions (2%-98% range)

* **Altman Z-Score:** Classic formula: Z = 1.2X1 + 1.4X2 + 3.3X3 + 0.6X4 + 1.0X5 with zone classification (Safe >2.99, Grey 1.81-2.99, Distress <1.81)

* **SHAP Explainability:** TreeExplainer provides feature-level contribution analysis showing which financial metrics drive the prediction

* **Market Intelligence:** Real-time news sentiment analysis and sector performance data

2. Machine Learning Module & Backend (Rohan & Jonathan)

Rohan and Jonathan developed the core prediction engine and API infrastructure that powers SolvencyInsight.

2.1 XGBoost Classification Model

The insolvency prediction model uses XGBoost (Extreme Gradient Boosting), an ensemble learning algorithm that builds decision trees sequentially, with each tree correcting errors from previous ones.

Model Configuration (insolvency_predictor.py):

```
self.model = xgb.XGBClassifier(
n_estimators=50, # Number of boosting rounds
max_depth=3, # Shallow trees prevent overfitting
learning_rate=0.05, # Small steps for gradual learning
reg_alpha=0.5, # L1 regularization (Lasso)
reg_lambda=1.0, # L2 regularization (Ridge)
min_child_weight=3, # Minimum samples per leaf
subsample=0.8, # Row sampling per tree
colsample_bytree=0.8 # Feature sampling per tree
)
```

Why These Parameters Matter:

* **n_estimators=50:** Limits model complexity; more trees can overfit on small datasets

* **max_depth=3:** Shallow trees capture general patterns without memorizing noise

* **reg_alpha and reg_lambda:** Regularization penalizes large weights, preventing any single feature from dominating

* **subsample and colsample_bytree=0.8:** Introduces randomness to improve generalization

2.2 Probability Smoothing

Raw XGBoost outputs often produce extreme probabilities (0% or 100%) which are unrealistic for financial predictions. We apply probability smoothing:

```
probabilities = 0.02 + 0.96 * raw_probabilities
```

This maps predictions to the range [2%, 98%], acknowledging that no model can be 100% certain about future bankruptcy.

2.3 SHAP (SHapley Additive exPlanations)

SHAP values are based on game theory - they measure each feature contribution to moving the prediction away from the baseline (average prediction). TreeExplainer is optimized for tree-based models.

```
explainer = shap.TreeExplainer(self.model)
shap_values = explainer.shap_values(X)
# Positive SHAP = increases bankruptcy risk
# Negative SHAP = decreases bankruptcy risk
```

2.4 Altman Z-Score Implementation

The Z-Score formula developed by Edward Altman in 1968 remains the gold standard for bankruptcy prediction:

```
Z = 1.2*(Working Capital/Total Assets)
+ 1.4*(Retained Earnings/Total Assets)
+ 3.3*(EBIT/Total Assets)
```

```
+ 0.6*(Market Value Equity/Total Liabilities)
+ 1.0*(Sales/Total Assets)
```

Zone	Z-Score Range	Interpretation
Safe	> 2.99	Low bankruptcy probability
Grey	1.81 - 2.99	Uncertain, requires monitoring
Distress	< 1.81	High bankruptcy probability

2.5 FastAPI Backend Architecture

The backend uses FastAPI, a modern Python web framework chosen for its automatic OpenAPI documentation, type validation via Pydantic, and async support.

Key Endpoints (main.py):

Endpoint	Method	Purpose
/api/financial/analyze	POST	Single company analysis with SHAP
/api/financial/upload	POST	Bulk CSV analysis
/api/financial/upload-single	POST	CSV upload with full SHAP response
/api/financial/feature-importance	GET	Model feature weights
/api/reports/insolvency	POST	Generate PDF report
/health	GET	System health check

2.6 Pydantic Schema Validation

All API inputs/outputs are validated using Pydantic models (schemas.py). This ensures type safety and automatic error messages for invalid data.

12 Financial Ratios Used: working_capital_to_total_assets, retained_earnings_to_total_assets, ebit_to_total_assets, market_value_equity_to_total_liabilities, sales_to_total_assets, current_ratio, quick_ratio, debt_to_equity, interest_coverage, net_profit_margin, return_on_assets, return_on_equity

3. Data Preparation & Frontend (Neha)

Neha handled data processing pipelines and contributed to the frontend implementation.

3.1 Data Processing Pipeline

Financial data requires careful preprocessing before model training. The pipeline handles:

* **Missing Value Handling:** Financial ratios with missing values are imputed using median values to preserve distribution

* **Feature Scaling:** StandardScaler normalizes features to zero mean and unit variance, critical for model convergence

* **CSV Parsing:** Pandas reads uploaded CSVs with automatic type inference and column validation

* **Data Validation:** Checks for required columns, valid numeric ranges, and data completeness

3.2 Training Data Generation

Synthetic training data was generated using domain knowledge about healthy vs. distressed companies:

```
# Safe companies: Higher ratios, stronger fundamentals
safe_data = {
working_capital_to_total_assets: np.random.uniform(0.15, 0.4),
current_ratio: np.random.uniform(1.5, 3.0),
debt_to_equity: np.random.uniform(0.2, 0.8), ...
}
# At-risk companies: Lower ratios, weaker fundamentals
risk_data = {
working_capital_to_total_assets: np.random.uniform(-0.1, 0.1),
current_ratio: np.random.uniform(0.5, 1.2),
debt_to_equity: np.random.uniform(1.5, 4.0), ...
}
```

3.3 Frontend Components Developed

FileUpload.tsx: Drag-and-drop CSV upload component with validation

* Accepts only .csv files with size validation

* Visual feedback during file selection

* Clear button to reset selection

DataTable.tsx: Sortable, searchable table for bulk results

* Client-side search filtering

* Column sorting with visual indicators

* Pagination for large datasets

3.4 API Service Layer (api.ts)

Neha implemented the Axios-based API client that connects frontend to backend:

```
const API_BASE_URL = import.meta.env.VITE_API_URL || '';

export const uploadSingleCompany = async (file: File) => {
const formData = new FormData();
formData.append('file', file);
const response = await api.post('/api/financial/upload-single',
formData, { headers: { 'Content-Type': 'multipart/form-data' }});
return response.data;
};
```

4. Frontend Development & UI/UX (Mariya)

Mariya designed and implemented the React frontend with focus on user experience and data visualization.

4.1 Technology Stack

* **React 18:** Component-based UI with hooks for state management

* **TypeScript:** Static typing for reliability and better developer experience

* **TailwindCSS:** Utility-first CSS framework for rapid styling

* **Recharts:** React charting library for SHAP visualizations

* **Vite:** Fast build tool with hot module replacement

4.2 InsolvencyAnalysis Page

The main analysis page (InsolvencyAnalysis.tsx) provides two modes:

Single Company Mode: Upload a CSV or manually enter 12 financial ratios. Results include risk gauge, Z-Score visualization, and SHAP chart.

Bulk Upload Mode: Process multiple companies via CSV. Results displayed in searchable/sortable table with summary statistics.

4.3 SHAP Visualization

The horizontal bar chart shows top 10 risk drivers:

* **Red bars:** Features that INCREASE bankruptcy risk (positive SHAP)

* **Green bars:** Features that DECREASE bankruptcy risk (negative SHAP)

* **Tooltip:** Shows actual feature value and SHAP contribution on hover

4.4 Risk Gauge Component

Custom gauge visualization showing probability of distress with color-coded zones (green/yellow/red) and animated needle indicator.

4.5 Responsive Design

The UI adapts to different screen sizes using Tailwind breakpoints (sm, md, lg, xl). Grid layouts adjust from single column on mobile to multi-column on desktop.

5. System Integration (All Team Members)

All four team members collaborated on integrating the components into a cohesive system.

5.1 Frontend-Backend Communication

Vite dev server proxies API requests to FastAPI backend:

```
// vite.config.ts
server: {
proxy: {
'/api': { target: 'http://localhost:8001', changeOrigin: true },
'/health': { target: 'http://localhost:8001', changeOrigin: true }
}
}
```

5.2 Data Flow Architecture

```
User uploads CSV --> FileUpload component --> api.ts (FormData) -->
FastAPI endpoint --> Pandas reads CSV --> XGBoost predicts -->
SHAP explains --> JSON response --> React state update -->
Charts render results
```

5.3 Error Handling

* **Frontend:** Try-catch with toast notifications for user feedback

* **Backend:** HTTPException with detailed error messages

* **Validation:** Pydantic models reject invalid input before processing

5.4 Model Metrics

Metric	Value	Meaning
Accuracy	~85%	Correct predictions overall
Precision	~82%	True positives among predicted positives
Recall	~88%	True positives among actual positives
F1-Score	~85%	Harmonic mean of precision/recall
AUC-ROC	~0.91	Model discrimination ability

6. Summary

SolvencyInsight demonstrates the practical application of machine learning to financial risk assessment. The system combines:

* **Traditional Finance:** Altman Z-Score provides interpretable baseline
* **Modern ML:** XGBoost captures non-linear patterns in financial data
* **Explainable AI:** SHAP values make predictions transparent and auditable
* **User-Friendly Interface:** React frontend makes complex analysis accessible

Team Contribution Summary:

Member	Components	Key Files
Rohan	XGBoost model, SHAP, Z-Score, Backend API	insolvency_predictor.py, main.py
Jonathan	Model training, regularization, API endpoints	insolvency_predictor.py, main.py
Neha	Data pipelines, CSV processing, API client	data files, api.ts, FileUpload.tsx
Mariya	React pages, charts, styling, components	InsolvencyAnalysis.tsx, RiskGauge.tsx